

* Local modules:- module that we create & use in our Application.

index.js

```
console.log('Hello world');
```

```
const add = (a,b) => {
  return a + b;
}
```

```
const sum = add(2,3);
console.log(sum);
```

```
% node index.js
Hello world
5
%
```

→ We create one fⁿ. called add(a,b) in our code & used it in some code.

But Now we try to create separate module for the same add(a,b) fⁿ. so we can use it any time...

→ For that we create file add.js in same folder in which our index.js is or we can say in our application folder...

> Vscode

add.js
index.js

add.js

```
const add = (a,b) => {
  return a + b;
};
const sum(4,5);
console.log(sum);
```

In node.js each file itself is module.

But it is isolated by default.

[i.e didn't invoked itself]

→ even if we made any error then it execution without inclusion can't noticed

→ In short if we create any module/file we must attach/include that module/file in our main flow execution.

→ Q. How we ask node to execute All modules?

→ & Here Common Js module comes in picture.

* CommonJs module structure:-

— CommonJs is a standard that stated how a module should be structured & shared.

— Node.js adopted CommonJs when it started out & is what we will see in code bases.

index.js

```
require('./add.js');
```

```
console.log("Hello");
```

→ indicates path of some/current folder in which index.js is.

→ require() fⁿ is always available in node - if included/attach the particular file/module from given path to current file → where you used it

Single/double as our preference.

```
% node index.js
```

```
4
Hello
%
```

— Here "require()" fⁿ loads the add module into its calling file eg. "index.js".

— The code in "add.js" will executed by V8 engine with "index.js" & if there is any console log then it will o/p on terminal.

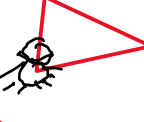
you can verify the sequence of execution in

Debug mode by:

placing the "break point" by clicking on line number.

— We will see "red dot" at the beginning of instruction line.

— Now Instead of running node in the terminal

"click on Debug & run icon  on side bar or select "Debug & Run" option from appropriate run menu

— After clicking on debug & run icon

— Select "Debug & Run" button & select "node.js"

here we are on first line of "index.js" & is at required('./add.js'); line

→ This starts the program in debug mode. with "debug & run" "toolbar" on top

— When we press/click on "step into" "↓" button & we can see require() which was at first line of "index.js" brings us to "add.js" - file/module.

— if we can "step over" "→" Tool-bar button - two times

— & we can see log-console o/p.

— step-over again & execution back again to next line after "require()" in index.js file/module.

→ step-over again & we can see "Hello" msg on console...

— This is how control flows with modules.

— last point instead of "add.js" you can also write only "add" in require fⁿ when requiring JS file

→ `require('./add.js');` is `require('./add');`

→ if we don't write extension ".js" - node automatically append extension ".js" to required file in require() fⁿ.

* Summary:- Local module -

— In node.js, each file is a module that is isolated by default.

— To load a module into another file, we use require() function.

— When main JS file (index.js) is executed, the code in the module is also executed.

— If the file we are requiring is a JavaScript file we can skip specifying the extension & node.js will infer it on our behalf.

→ Now Although we are able to load & execute the module what would be better as if could expose certain functionality of module/files by keeping the rest private

— That would allowed us to share only the necessary codes between modules.

— how? — - ->